Taylor Mohney

Las Vegas, Nevada, United States   TaylorMohney@icloud.com   702-376-4552   in/taylormohney
github.com/CatsMeow492   youngmohney.com

SUMMARY

Senior Software Engineer with 10+ years shipping production **AI and ML products**—from customer-support chatbots and agentic developer tooling to large-scale ML serving infrastructure. Strong in **TypeScript, Python, and Go**, with deep experience delivering **LLM-powered experiences to non-technical end users** and operating them reliably on **Docker, Kubernetes, and the cloud (AWS & GCP)**. Skilled in **prompt and context engineering, multi-agent orchestration, retrieval-augmented generation (RAG), production eval loops**, and the **human-in-the-loop UX patterns** that make AI features trustworthy in the field.

EXPERIENCE

Principal Engineer
Kindly Robotics — June 2025 – Present, Remote

- **Kindly IDE—agentic IDE for robotics engineers.** Shipped an LLM-powered development environment (Cursor-style) purpose-built for robotics: chat-over-codebase with awareness of URDF, ROS launch files, and calibration packs; inline config completions; an agent mode that scaffolds and validates whole robot configurations end-to-end. End users are robotics engineers, not ML practitioners—UX had to hide the model entirely when it worked and fail legibly when it didn't.
- **Multi-agent robot configuration framework.** Architected a planner/generator/validator agent graph that turns a natural-language spec (e.g. "6-DOF arm with gripper for pick-and-place at station 4") into a validated configuration—URDF, launch files, calibration—cycled through a simulation-in-the-loop verifier before the engineer ever sees it; work that took days of manual config lands in minutes.
- **Retrieval over robotics knowledge.** Built a RAG pipeline over internal design docs, past customer configs, and manufacturer specs so IDE suggestions are grounded in what the team has actually shipped rather than what the base model guessed, with citation surfacing so engineers can trust but verify.
- **Production eval harness for generated configs.** Set up an offline eval loop that replays real engineer tasks against each model version, scoring compile-pass rate, simulation-success, and engineer override rate—catching regressions before they reach users and informing prompt and retrieval-pipeline changes.
- **Defense & industrial ML integration.** Partnered with defense contractors to embed ML pipelines—predictive maintenance, anomaly detection, adaptive process control—into existing production workflows, enabling real-time operational visibility, higher line efficiency, and lower defect rates at scale.

Lead Full Stack Developer
Department of Defense — February 2024 – May 2025, Remote

- **Container-native model serving.** Deployed computer-vision models as serverless endpoints with KServe on Kubernetes, packaged by Helm charts that offered rollbacks and environment-specific values; achieved ≤ 200 ms p95 latency even under surge traffic.
- **Automated train-to-serve pipeline.** Orchestrated Kubeflow Pipelines for data prep & training, then handed artifacts to Argo Workflows for CI/CD canary releases—cutting the model promotion cycle by 40 %.
- **Experiment tracking & model registry.** Instituted MLflow for run-level metrics, artifacts, and versioned model stages, enabling reproducible A/B tests across GPU and CPU targets.
- **Rigorous performance benchmarking.** Integrated NVIDIA Triton Inference Server and its *perf_analyzer* to tune dynamic batching; reduced average inference latency by 25 % and maximized GPU utilization.
- **Production observability & drift defense.** Exported real-time throughput/latency from Triton to Prometheus, surfaced via Grafana dashboards, and scheduled nightly Evidently AI reports for data- and concept-drift alerts with auto-rollback on drift > 0.3.
- **Secure, zero-downtime shipping.** Shipped every micro-service as a multi-arch Docker image; Helm's blue-green deployments delivered seamless upgrades and instant rollbacks across classified & unclassified clusters.
- **Edge AI inside WebRTC mesh.** Extended the peer-to-peer video platform by side-car-deploying TorchServe containers that ran real-time face-recognition filters via the WebRTC Transform API—adding < 50 ms overhead while preserving 99.9 % uptime.
- **Analyst-facing review surface.** Shipped the Next.js application operators used to triage CV model detections, correct labels, and feed corrections back into retraining—closing the human-in-the-loop label → retrain → redeploy cycle in minutes instead of weeks. Secured with modular authentication and role-based access across classified and unclassified environments.

Senior Full Stack Developer – Angular, React, Node, Python
USDA — January 2022 – February 2024, Remote, Missouri

- Led design and implementation of scalable, data-driven applications used daily by USDA program staff and field personnel, reducing software bugs by 35% and accelerating feature delivery timelines by 20%.
- Designed and developed microservices using Angular and FastAPI, integrating with AWS Lambda and API Gateway to enable seamless communication between architectural components.
- Streamlined backend processes by developing Python-based APIs, enhancing system performance and reliability, resulting in a 25% reduction in latency for key workflows.
- Migrated critical features to a modular architecture using Angular, React, and AWS services, improving maintainability and deployment efficiency by 40%.
- Engineered end-to-end web solutions leveraging Angular, React, Node, and Python, leading to a 50% increase in user engagement and a 30% boost in application responsiveness.

Full Stack/Web3 Developer – Solidity, React, Node, AWS
Coinbase May 2021 – December 2022, Remote, New York

- **Coinbase NFT marketplace UX (React + TypeScript).** Led development of user-facing listing, discovery, and checkout flows for non-technical traders—wallet-aware browsing, live price updates via Blocknative websockets, and graceful error handling for the failure modes unique to on-chain UX. Routed high-value listings through a private RPC whenever an in-house LightGBM risk score exceeded 0.7, giving users front-run-protected execution without asking them to understand what MEV is—trimming slippage ∼15 bps at 99.9 % UX uptime.
- **Built an in-house "MEV Sentinel" platform**—a Rust/Go micro-service that live-streams raw transactions from the public mempool and Flashbots' relay API, feeds them through Bloom-filter pre-screens for sandwich / back-run signatures, **then scores each bundle with a LightGBM ensemble trained on 300 k labeled blocks**. Models were tracked in MLflow's Model Registry for versioned promotion to *Staging* and *Production*, containerised with Docker, and served at <200 ms p99 via KFServing/KServe on Kubernetes, rolled out by Helm charts.
- **Devised a post-block forensics pipeline** by extending *mev-inspect-py* with custom SQL models and Grafana dashboards. Daily ETL jobs replay validator payments and arbitrage flows, while a PyTorch Lightning notebook retrains the LightGBM ensemble weekly for concept-drift resistance.
- **Authored a mempool replay harness (Node.js + Hardhat + Next.js)** that forks mainnet state for deterministic simulation of pending bundles, letting the team regression-test contract upgrades against new ML-flagged MEV vectors—**cutting exploit incidents to zero last year**.
- **Instrumented Solidity smart contracts with MEVProof events** that emit gas-price and tx-origin fingerprints. Logs stream through AWS Kinesis → Lambda and land in Redshift for near-real-time Evidently AI drift reports; any AUC drop below 0.85 auto-triggers a Helm rollback of the KServe deployment.
- **Integrated DeFi yield strategies only after LightGBM stress-tests**, ensuring auto-compounding and token-swap flows avoided toxic back-runs—boosting user volume 25 % at <1 % failed-tx rate.
- **Modified an execution-layer geth fork** to randomise tx ordering within gas-price bands, empirically showing (via the MLflow offline evaluation suite) a 25 % drop in miner-side MEV capture compared to baseline.
- **Shipped a real-time React + TypeScript operator dashboard** that surfaces model inference latency, false-positive rate, and live MEV extraction metrics—powered by Prometheus exporters embedded in the KServe pod and visualised in Grafana.

Contract Software Engineer
Apple November 2020 – May 2021, Remote

- **Customer-support chatbots at scale.** Built and shipped conversational AI for customer support—intent classification and entity extraction on Hugging Face Transformers (BERT, DistilBERT) combined with OpenAI GPT-3 (beta) for open-ended response generation, gated behind confidence thresholds and escalation to human agents for low-confidence turns.
- **Owned the end-to-end conversation experience.** Designed dialogue flows, tool integrations to order, account, and device lookup APIs via GraphQL on Node.js, and handoffs to human agents—optimising for deflection rate and customer satisfaction, not just model accuracy.
- **Retrieval-grounded answers.** Stood up embedding search over support-knowledge content using Hugging Face sentence encoders and a vector index so the bot could cite a real policy doc instead of hallucinating—an in-house RAG pattern before the term was mainstream.
- **ML CI/CD and production observability.** Containerised model and dialogue services with Docker; CI/CD via GitHub Actions and Terraform on AWS (SageMaker endpoints, ECS), reducing deployment friction and model rollback time by 40%. Instrumented Prometheus and Grafana to track response latency, confidence distribution, abandonment, and escalation-to-human rate—pairing ML metrics with product metrics to tune thresholds and fallback copy.
- **Human-in-the-loop iteration with non-technical users.** Partnered with support ops, ML engineers, and front-end teams to roll out chat experiences to non-technical customers, iterating on guardrails, refusal behaviour, and error copy as failure modes surfaced in production—cutting integration delays by over 30 %.

Full Stack Developer

Ai-Blockchain December 2017 – November 2020, Remote, California

- Designed and implemented a scalable architecture for blockchain-based payment systems, enhancing operational efficiency and reducing downtime by 40%.
- Conducted rigorous code reviews and established best practices for secure smart contract development, preventing vulnerabilities and enhancing code quality.
- Collaborated with cross-functional teams to integrate blockchain technology with existing payment gateways, streamlining adoption for businesses.
- Enhanced system reliability through automated testing pipelines, achieving a 90% reduction in critical bugs post-deployment.
- Spearheaded adoption of emerging blockchain consensus protocols, improving transaction validation speed and network security.

Full Stack Developer Intern

Zenbase December 2016 – December 2017, Remote

- Implemented a far-reaching rental payment platform feature using React and Node.js, enhancing user experience and increasing transaction efficiency by 25% within three months.
- Implemented Solidity contracts for a proof of meditation platform, ensuring smart contract security and functionality using industry best practices.
- Developed smart contracts in Solidity, enhancing gas efficiency through the use of bit manipulation and inline assembly, resulting in a 15% reduction in transaction costs.
- Utilized modern tools to detect and address security vulnerabilities, enhancing platform security by 40%.

EDUCATION

Computer Science

University of Nevada Las Vegas · Las Vegas, Nevada · 2019

CERTIFICATIONS

AWS Machine Learning Certification – Specialty

Amazon Web Services · 2025

- Certified in Machine Learning and Artificial Intelligence training and application with AWS technologies
- Certified in data preparation and analysis/science with AWS technologies

AWS Certified DevOps Engineer – Professional

Amazon Web Services Training and Certification · 2025

- Certified in AWS DevOps technologies and applications

AWS Certified Developer – Associate

Amazon Web Services Training and Certification · 2023

- Certified in development with all AWS technologies
- Application of Cloud and Serverless technologies

AWS Certified Cloud Practitioner

Amazon Web Services Training and Certification · 2022

- Certified in AWS cloud technologies and their application

SKILLS

AI/ML Applications: LLM integration (OpenAI, Anthropic, open-weights via vLLM & Ollama), prompt & context engineering, retrieval-augmented generation (RAG), multi-agent orchestration, tool use & function calling, conversational UX, human-in-the-loop systems, production eval pipelines

ML Infrastructure: Hugging Face Transformers, KServe/KFServing, NVIDIA Triton, TorchServe, MLflow, Kubeflow Pipelines, Argo Workflows, Ray, Evidently AI, Prometheus, Grafana

Front End: HTML, CSS, TypeScript, JavaScript, React, Angular, Next.js

Backend: Go, Python, JavaScript, TypeScript, Node, Express, FastAPI, Django, GraphQL

Blockchain: Rust, Solidity, Web3.js

Research: Adaptive LoRA Placement for Efficient Large Language Model Fine-tuning; Efficient Compression of Large Language Models via Adaptive Quantization; Parameter-Efficient Fine-tuning of Large Models: A Comprehensive Analysis

DevOps: Docker, Kubernetes, AWS (ECR, ECS, EKS, Lambda, API Gateway, EC2, ALB, IAM), Azure, GCP, Terraform, GitHub Actions, Git